9.6. Números en Punto Flotante.

Se denominan así a las representaciones internas al procesador que modelan a los números reales. En forma externa, se representan como números con coma (3.1415926 con punto), o bien en notación científica $0.12\ 10^{-5}$ (con un simple dígito a la izquierda del punto decimal).

Se dice que el número está normalizado si el dígito a la izquierda no es cero; en el ejemplo anterior: $1.2\ 10^{-6}$. En el caso de computadores, se emplea números en sistema binario, y con un número finito de dígitos.

Existe un compromiso entre los bits dedicados al exponente y los que representan la mantisa(o cifras significativas del número). El compromiso es entre el rango de representación y la exactitud de la representación. Por otro lado, debido a la forma de accesar la memoria, los números deben ser una o más palabras de la memoria.

9.6.1. Norma IEEE 754.

Estudiaremos números de punto flotante de acuerdo a la norma IEEE 754. El estándar que se impuso a las diferentes formas que se intentaron históricamente.

En forma externa, un número flotante normalizado, se interpreta:

$$(-1)^{S}*1.M_2*2^{Ee}$$

La mantisa siempre comienza en 1, y M representa un fraccionario puro sin signo.
Ee es el exponente representado en forma externa.

En forma interna, ocupando 32 bits, se tiene el número punto flotante precisión simple(en C es el tipo float):
Donde S = 0 representa números positivos, S=1 representa números negativos.

Signo	**Exponente**	Mantisa normalizada simple
1	8	23

Después del signo, se coloca primero el exponente, para poder comparar números. Ya que a mayor exponente, mayor es el número.

También se tiene el tipo double, el que en forma interna, se representa:

Signo	**Exponente**	Mantisa normalizada doble
1	11	20+32

Ocupa dos palabras consecutivas, o se requieren dos registros.

El exponente Ei, en forma interna, se representa como número polarizado con signo.
Para 8 bits: $-127 <= Ee <= +127$ con $Ei = Ee + 127$

Para 11 bits: -1023 <= Ee <= +1023 con Ei = Ee + 1023
Esta notación también favorece la comparación. Los exponentes quedan ordenados de más negativos a más positivos, en forma ascendente. (el exponente menor es la secuencia de puros ceros).

La norma IEEE 754 contempla: números normales, sub-normales, representaciones para el cero y el infinito, y para números inválidos (NaN, not a number).

Con –127 = Ee y M_2 !=0	Ei=0	se tiene $N = (-1)^S*0.M_2*2^{-126}$	Números sub-normales.
Con –127 = Ee y M_2 ==0	Ei=0	se tiene $N = (-1)^S*0.0$	Cero(con signo).
Con –127 < Ee <=127		se tiene $N = (-1)^S*1.M_2*2^{Ee}$	Números normales.
Con Ee =-128 y M_2 ==0	Ei= -1	se tiene $N = (-1)^S * \infty$	Infinito(con signo).
Con Ee =-128 y M_2 !=0	Ei= -1	se tiene N = NaN	Número inválido.

Rango de representación. (precisión simple)
Si el número a representar es menor que el menor representable, se dice que existe underflow (vaciamiento).

El menor representable es un número no normalizado.

La representación interna para el menor positivo es: 0x00000001

S = 0, Ei = 0, M = $00000000000000000000001_2$ Como M es diferente de cero y Ei es cero, se tendrá un número sub-normal, que se interpreta según: $N = (-1)^S*0.M_2*2^{-126}$
$+ 0.00000000000000000000001_2 *2^{-126} = 2^{-126-23} = 1.401298464324817e-45$

Entonces se tiene vaciamiento para $|N| < (-1)^S*2^{-149} \approx 1{,}40130\ 10^{-45}$

Si el número a representar es mayor que el mayor representable se dice que existe overflow (rebalse).

El positivo mayor representable tiene: cero en el signo, el mayor exponente positivo (+127), y la mayor mantisa (ésta es el uno implícito, con puros unos en los bits dedicados a la mantisa; estos unos representan un valor menor que uno, y con el uno implícito se obtiene el dos como cota superior aproximada).

$0x7f7fffff = + 1.11111111111111111111111_2 *2^{127} = 3.4028234663852886e+38$

Rebalse para $|N| > (-1)^S*(2- 2^{-23})*2^{127} \approx 3{,}402823\ 10^{38}$

Para obtener las cifras decimales: $2^{127} = 10^x$, que implica:
$x = 127 \log 2 = 127 * 0{,}30103 \approx 38$.

Nótese que no pueden representarse números muy pequeños, ni números muy grandes.

La representación interna de +0.0, por convenio es una secuencia de puros ceros.

En doble precisión, con 11 bits, en decimal el exponente mayor es aproximadamente 308; el menor –308.

Constantes.

Se incluyen algunas definiciones de constantes relacionadas con números en punto flotante que se encuentran en el archivo float.h de la biblioteca de C.

```
#define FLT_RADIX        2
#define FLT_ROUNDS       1
#define FLT_GUARD        1
#define FLT_NORMALIZE    1

#define DBL_DIG      15          /*dígitos decimales de un double  */
#define FLT_DIG       6          /*dígitos decimales de un float   */

#define DBL_MANT_DIG    53   /* bits de la mantisa de un double  */
#define FLT_MANT_DIG    24   /* bits de la mantisa de un double  */

#define DBL_EPSILON      2.2204460492503131E-16
/* el mínimo espacio entre dos números doubles representables */
#define FLT_EPSILON      1.19209290E-07F
/* el mínimo espacio entre dos números float representables. Equivale a 2^-23 */
/* menores positivos normales */
#define DBL_MIN          2.2250738585072014E-308     /* módulo double menor*/
#define FLT_MIN          1.17549435E-38F         /* módulo float menor. Note la F.*/
```

Ejemplos.

a) Convertir N= –0,75 a representación interna.

$-0{,}75_{10} = -(0.5 + 0.25)_{10} = -(2^{-1} + 2^{-2})_{10} = -0.11_2$ y normalizado: $-1.1 * 2^{-1}$

Entonces: S=1, ya que el signo es negativo.

M = 100 0000 0000 0000 0000 0000; debe recordarse que el primer uno no se coloca en la representación interna de la mantisa. Esta es una fracción pura.

Ee= -1

Ei = Ee + 127 = -1 + 127 = 126_{10} = $0111\ 1110_2$

Resulta N = $1011\ 1111\ 0100\ 0000\ 0000\ 0000\ 0000\ 0000_2$ = 0xBF400000

La conversión no es tan simple, si el número no puede descomponerse en sumas de potencias negativas de dos, menores que 23. O si el número es irracional, o si es una fracción periódica. En estos casos se aplica divisiones sucesivas.

La representación de N= 1/3 en precisión simple

$+ 1.01010101010101010101010_2 * 2^{-2} = 0.3333333134651184$

Existe un error por truncamiento, ya que no es posible expresar en forma exacta con 23 bits para la mantisa.

En doble precisión, se tiene más cifras en la mantisa, pero aún existe truncamiento.
$+ 1.0101010101010101010101010101010101010101010100011001 * 2^{-2} = 0.33333333333333$

b) Convertir a representación externa:

0	01101000	101 0101 0100 0011 0100 0010

S = 0, implica signo positivo.
Ei = 0x68 = 104 , implica Ee = 104 – 127 = -23
$M= 2^{-1} +2^{-3} +2^{-5} +2^{-7} + 2^{-9} + 2^{-14} +2^{-15} +2^{-17} +2^{-22} = 0.666115$
La secuencia representa a: $+ 1.666115 * 2^{-23} \approx 0{,}000000198616385.. \approx 1{,}986164E-7$

c) Convertir a representación externa (el menor normal representable. FLT_MIN):

0	00000001	000 0000 0000 0000 0000 0000

S = 0, implica signo positivo.
Ei = 0x01 = 1 , implica Ee = 1 – 127 = -126
M = 000 0000 0000 0000 0000 0000
La secuencia representa a:
$+ 1.00000000000000000000000_2 * 2^{-126} = 1.1754943508222875e-38$

d) El menor número sub-normal, representado en forma interna en hexadecimal, es 0x00000001.

S = 0, implica signo positivo.
Ei = 0x00 = 0 , implica Ee = 0 – 127 = -127 Implica número sub-normal
$M = 0.00000000000000000000001_2$ Entonces:
$+ 0.00000000000000000000001_2 * 2^{-126} = 1.401298464324817e-45$ (sub-normal)

e) El cero, se representa, según:

0	00000000	000 0000 0000 0000 0000 0000

f) El valor asociado a la constante FLT_EPSILON.

Convertir a representación externa (el mínimo espacio entre dos representables):

0	01101000	000 0000 0000 0000 0000 0000

S = 0, implica signo positivo.
Ei = 0x68 = 104 , implica Ee = 104 – 127 = -23
M = 0
La secuencia representa a:
$+ 1.00000000000000000000000_2 * 2^{-23} \approx 1.1920928955078125e-7 \approx 0{,}000000119$

La mantisa no normalizada equivale a: $.000\ 0000\ 0000\ 0000\ 0000\ 0001_2$

Esto muestra porque los flotantes de precisión simple permiten representar números decimales con seis cifras decimales significativas, ya que dos números adyacentes difieren en FLT_EPSILON.

g) El mayor sub-normal positivo es:
$+ 0.11111111111111111111111 * 2^{-126} = 1.1754942106924411e-38$

Llamados al sistema en SPIM para leer y escribir flotantes.

El siguiente programa ilustra los llamados al sistema para leer y escribir un número en punto flotante en la consola de SPIM. Los números pueden ingresarse con punto decimal, con y sin exponente decimal. En forma similar a la sintaxis de C (1.75 con punto decimal, y empleando notación exponencial 0.5e-3. Debe ser e minúscula).

El programa permite experimentar con la conversión de números punto flotante, en representación externa decimal, a representación interna formato IEEE 754, con apoyo del simulador. Estas operaciones son tediosas de realizar con papel y lápiz.

```
                .data
mensaje2:       .asciiz  "\nEntre un número real  = "
x:              .float  0.0
                .text
                .globl   main
main:
                li       $v0,4
                la       $a0,mensaje2
                syscall
                li       $v0,6          #read float en f0
                syscall
                s.s      $f0,x          #store single en $f0 en variable estática x.
                                        #macro que emplea swc1
                mov.s    $f12,$f0       #argumento en $f12
                li       $v0,2          #print float
                syscall
                j        main
```

Las operaciones en punto flotante requieren de algoritmos especiales y de un hardware dedicado (desde 1990 se emplea un coprocesador integrado al procesador en el mismo chip, con registros independientes, y que puede comunicar los resultados a la memoria).

Simulador de representación y operaciones con números.

Existe en la red un simulador de algoritmos aritméticos para computadores, en la siguiente dirección:
http://www.ecs.umass.edu/ece/koren/arith/simulator/

En el simulador se pueden visualizar las representaciones internas y externas de números en complemento base y flotantes, en las operaciones aritméticas (sumar, restar, multiplicar, dividir).

9.6.2. Algunas operaciones aritméticas en punto flotante.

Suma.

Se describe la operación para **sumar** dos números en punto flotante.
Primero se desplaza la mantisa del número menor hacia la derecha hasta que su exponente sea igual al del número mayor. Esto puede hacer que se pierdan cifras de representación.
Luego se suman las mantisas. El resultado puede no quedar normalizado.
Se normaliza el número variando el exponente, y debe verificarse que no ocurra rebalse o vaciamiento de éste.
Después de la normalización, puede ser necesario redondear la mantisa al número de bits de la representación.
Podría ser que al redondear, el número se desnormalice, en ese caso debe repetirse el paso de normalización.

Lo usual es dedicar hardware especial para implementar el algoritmo anterior. En caso de no tenerlo, la solución es implementar una biblioteca, en base a las operaciones de la unidad aritmética entera, que realice el algoritmo anterior. En este caso, obviamente la realización de una simple suma de flotantes requiere la ejecución de varias instrucciones, alargando el tiempo de ejecución de los programas que empleen números reales.

Multiplicación.

La operación de **multiplicación** en punto flotante, puede describirse según:
Primero se suman los exponentes. En esto debe considerarse que los exponentes están polarizados; por lo tanto debe restarse la polarización para obtener el exponente correcto. En decimal: si un exponente es 5(polarizado en 127 es 5 + 127= 132) y el otro exponente es -7(polarizado en 127 es -7 + 127= 120), la suma de los exponentes resulta 132+120 = 252, que no es el exponente correcto. Debe realizarse: 252-127 = 125 (que equivale a -2).
Luego se multiplican las mantisas.
Se normaliza el resultado, corriéndolo a la derecha e incrementando el exponente. Verificando si se produce rebalse.
Redondear la mantisa, verificando que el resultado esté normalizado; en caso de no estarlo repetir el proceso de normalización.
Finalmente debe colocarse el signo.

La aritmética de punto flotante tiene un alto grado de elaboración, una de las refinaciones es el redondeo, esto cobra especial importancia en este sistema numérico que en sí es una aproximación del número real. Se emplean bits adicionales denominados de guarda y redondeo en los cálculos intermedios.

El algoritmo de división, es bastante más complejo, y produce una operación más lenta. Para acelerarla en algunos diseños se genera el recíproco; en otros se intenta obtener varios bits de la división en un solo paso. Un pequeño error en la implementación de este algoritmo causó una pérdida millonaria a la empresa Intel, en el diseño del procesador PENTIUM.

La norma IEEE 754, tiene símbolos especiales para representar más y menos infinito, y lo que denomina NAN (not a number) que ocurre en la división de cero por cero, o en una resta de infinito menos infinito. También se tratan números no normalizados que permiten extender el rango de representación.
Más detalles sobre los temas anteriores pueden encontrarse en el texto guía.

Las operaciones en punto flotante requieren de algoritmos especiales y de un hardware dedicado (tradicionalmente como un coprocesador, con registros independientes, que puede comunicar los resultados a la memoria)

Instrucciones MIPS en punto flotante.

En el caso del MIPS, el coprocesador de punto flotante es el número 1.
Existen 32 registros de punto flotante de 32 bits cada uno. Para manipular doubles se emplean registros pares; el par más el impar siguiente forman el espacio para el doble. Más adelante se desarrollan ejemplos en assembler para la manipulación de flotantes.

A continuación se ilustran algunas instrucciones de movimiento de datos, (en el manual del programador figura el repertorio completo de instrucciones y macro instrucciones para tratar con números reales).

```
lwcz    Reg, 0($t2)          #Load Word Coprocessor
#Load the word at address into register Reg of coprocessor z (0..3).
swcz    Reg, 0($t2)          #Store Word Coprocessor
#Store the word from register Reg of coprocessor z at address.

mfcz    CpuSrc, CPdest       #Move CPdest From Coprocessor z
#Move the contents of coprocessor z 's register CPdest to CPU register CpuSrc.
mtcz    CpuSrc, CPdest       #Move To Coprocessor z
#Move the contents of CPU register CpuSrc to coprocessor z 's register CPdest.

bczt    label                #Branch to label if Coprocessor z flag is set (True)
bczf    label                #Branch to label if Coprocessor z flag not set (False)
#Se emplean después de comparaciones en punto flotante
```

Ejemplos de programación assembler con flotantes.

Suma de dos números.

El siguiente segmento en C, produce la suma de dos números en punto flotante.

```
#include <stdio.h>
 float  x = 1.1; float  y = 2.2; float  z1, z2;
int main (void)
 {       z1 = x + y;     printf("%f\n",z1);
         z2 = x * y;     printf("%f\n",z2);
         return(0); }
```

Se traslada al siguiente código assembler:

```
                .data
x:              .float  1.1       #Variables en segmento de datos
y:              .float  2.2       #  0.22e+1  en notación científica.
z1:             .float  0.0
z2:             .float  0.0
newline:        .asciiz "\n"
                .text
                .globl  main
main:
                la      $t0, x            #  t0 = &x
                lwc1    $f0, 0($t0)       #  f0 = x          (lectura desde memoria)
                la      $t0, y
                lwc1    $f2, 0($t0)       # f2 = y
                add.s   $f4, $f0, $f2     # f4 = x + y
                la      $t0, z1           # t0 = &z1
                swc1    $f4, 0($t0)       # *t0 = f4    z1 = x + y  (escritura en memoria)

                mov.s   $f12, $f4         #argumento en $f12
                li      $v0,2             #print float
                syscall
                la      $a0, newline      #argumento en $a0
                li      $v0,4             #print string
                syscall

                mul.s   $f4, $f0, $f2     # f4 = x * y
                la      $t0, z2           # t0 = &z2
                swc1    $f4, 0($t0)       # *t0 = f4    z2 = x * y

                mov.s   $f12, $f4         # argumento en $f12
                li      $v0, 2            # print float
                syscall
                la      $a0, newline      # argumento en $a0
```

```
        li      $v0, 4          # print string
        syscall

        li      $v0, 10                 # exit   Retorno al monitor.
        syscall
```

Notar que se emplean registros pares para flotantes en simple precisión.

El llamado número 10 al sistema retorna a la primera instrucción del trap handler, que opera como un monitor(o sistema operativo primitivo).

Conversión.

El siguiente segmento en C, convierte de grados Farhenheit a Celsius.

```
#include <stdio.h>
#define ctef1   5.0/9.0
#define ctef2   ctef1*32.0
/*  celsius = (5/9)*(farh -32) = ctef1*farh-ctef2  */
 float  farh= 100.0;
 float  celsius;

int main (void)
 {  celsius = ctef1*farh-ctef2;
        printf("%f\n",celsius);
        return(0);
 }
```

Se emplean constantes con órdenes #define del preprocesador.

El ensamblador MIPS, del simulador SPIM, no permite el uso de constantes. Se almacenarán las constantes en la zona estática de datos.

```
        .data
ctef1:  .float  0.55555556   #5/9 redondeado a 8 cifras
ctef2:  .float  17.777778    #(5/9)*32 redondeado a 8 cifras.
farh:   .float  100.0        #
celsius:.float  0.0
#El valor de celsius, con calculadora Windows, es 37.77777777777777777778
        .text
        .globl  main
main:
        la      $t0, ctef1      # t0 = &ctef1
        lwc1    $f0, 0($t0)     # f0 = *t0
        la      $t0, ctef2
        lwc1    $f2, 0($t0)     # f2 = ctef2
        la      $t0, farh
```

```
lwc1    $f4, 0($t0)     # f4 = farh
mul.s   $f6, $f4, $f0   # f6 = farh * ctef1
sub.s   $f8, $f6, $f2   # f8 = farh * ctef1 - ctef2
la      $t0, celsius    # t0 = &celsius
swc1    $f8, 0($t0)     # *t0 = f8

mov.s   $f12, $f8       #argumento en $f12
li      $v0, 2          #print float
syscall

li      $v0, 10                 #exit
syscall
```

La secuencia:

```
la      $t0, ctef1      # t0 = &ctef1
lwc1    $f0, 0($t0)     # f0 = *t0
```

Puede reemplazarse por la macro instrucción: l.s $f0,ctef1

La secuencia:

```
la      $t0, celsius    # t0 = &celsius
swc1    $f8, 0($t0)     # *t0 = f8
```

Puede reemplazarse por la macro instrucción: s.s $f8,celsius

Empleo de doble precisión.

Para emplear aritmética de punto flotante en doble precisión, se estudia el siguiente ejemplo:

```
#include <stdio.h>
#define ctef1   5.0/9.0
#define ctef2   ctef1*32.0
/*  celsius = (5/9)*(farh -32) = ctef1*farh-ctef2  */
 double  farh= 100.0;
 double  celsius;

int main (void)
 {  celsius = ctef1*farh-ctef2;
        printf("%2.17f\n",celsius);
        return(0);
 }
```

Note la especificación de dos cifras enteras y 17 decimales en el argumento de control de printf. Al ejecutar este programa en C, podrá advertirse los valores de las últimas cifras.

Al pasar a assembler, note el número de cifras para especificar, con la mayor precisión que es posible las constantes dobles (16 cifras decimales, redondeadas)

```
.data
```

```
ctef1:      .double   0.5555555555555556#5   #5/9
ctef2:      .double  17.777777777777778#7    #(5/9)*32
farh:       .double  100.0        #
celsius:    .double  0.0          #37.77777777777777777777778
            .text
            .globl   main
main:
            l.d      $f0, ctef1       #macro
            l.d      $f2, ctef2
            l.d      $f4, farh
            mul.d    $f6, $f4, $f0    # f6 = farh * ctef1
            sub.d    $f8, $f6, $f2    # f8 = farh * ctef1 - ctef2
            s.d      $f8, celsius
            mov.d    $f12, $f8        #argumento en $f12
            li       $v0, 3           #print double
            syscall

            li       $v0, 10                  #exit
            syscall
```

La macro instrucción: l.d $f0, ctef1 se expande en la secuencia:

```
            la       $t0, ctef1
            lwc1     $f0, 0($t0)
            lwc1     $f1, 4($t0)
```

Note cómo se emplea el par de registros $f0, $f1 para almacenar el flotante doble precisión. Es similar la macro instrucción s.d (por store double).